Angela Wei

Data 260

Homework 8

Section 0:

Hardware Used: HP Pavilion Windows 11 Pro

Local Models Used: Docker, Kafka, localhost at PORT_BASE 8439, FastAPI

Commit Hash: All commits labeled “Homework 8 Submission” starting with 823166391736c5a4024ac24d043f9889810949ac

SID4: 1339

PORT_BASE: 8439

PREFIX: s1339

SEED: 1339

VERIFY_SEED: 261339

DOMAIN_ID: 3 – Grocery Supply and Recall Notices

Part 1

A process_request function implemented in hw7-authentication.py file from previous assignment (to build upon it). Here CREATE_GROCERY_RECALL_NOTICE and GET_GROCERY_RECALL_NOTICE are implemented.

```python
def process_request(request, notices):
    correlation_id = request.get("correlation_id")

    def failure(message):
        return {"correlation_id": correlation_id, "success": False, "error": message}

    if not isinstance(correlation_id, str) or not correlation_id.strip():
        return failure("Missing or invalid correlation_id")
    if not isinstance(request.get("reply_to"), str) or not request["reply_to"].strip():
        return failure("Missing or invalid reply_to")
    operation = request.get("operation")
    if operation not in ("CREATE_GROCERY_RECALL_NOTICE", "GET_GROCERY_RECALL_NOTICE"):
        return failure("Unknown operation")
    data = request.get("data")
    if not isinstance(data, dict):
        return failure("data must be an object")

    if operation == "CREATE_GROCERY_RECALL_NOTICE":
        try:
            item = GroceryRecallNoticeCreate.model_validate(data)
        except ValidationError as exc:
            return failure(str(exc))
        if any(n["unique_code"] == item.unique_code for n in notices.values()):
            return failure("Grocery recall notice with this unique_code already exists")
        notice_id = str(uuid4())
        notices[notice_id] = {"grocery_recall_notice_id": notice_id, **item.model_dump()}
        return {
            "correlation_id": correlation_id,
            "success": True,
            "grocery_recall_notice_id": notice_id,
            "message": "Grocery recall notice created",
        }
```

```python
    notice_id = data.get("grocery_recall_notice_id")
    if not isinstance(notice_id, str):
        return failure("Missing or invalid grocery_recall_notice_id")
    try:
        UUID(notice_id)
    except ValueError:
        return failure("grocery_recall_notice_id must be a UUID")
    if notice_id not in notices:
        return failure("Grocery recall notice not found")
    return {"correlation_id": correlation_id,
            "success": True,
            "grocery_recall_notice": notices[notice_id]}
```

A run_worker function implemented in the hw7_authentication.py file. This functions as the Kafka worker service that creates the connection to process_request() by creating a consumer for the request.

```python
def run_worker():
    servers, requests_topic, responses_topic = kafka_settings()
    consumer = KafkaConsumer(
        requests_topic, bootstrap_servers=servers,
        group_id=f"{requests_topic}_worker", auto_offset_reset="earliest",
        enable_auto_commit=False,
    )
    producer = make_producer(servers)
    notices = {}
    completed = {}
    try:
        ready = False
        while True:
            batches = consumer.poll(timeout_ms=1000, max_records=1)
            if consumer.assignment() and not ready:
                log.info("Consumer is ready and listening...")
                ready = True
            for message in (m for batch in batches.values() for m in batch):
                try:
                    request = json.loads(message.value)
                except (ValueError, UnicodeDecodeError):
                    log.error("WORKER invalid JSON; no usable reply envelope")
                    consumer.commit()
                    continue
                if not isinstance(request, dict):
                    log.error("WORKER request must be an object")
                    consumer.commit()
                    continue
                log.info("WORKER RECEIVE %s", request)
                cid = request.get("correlation_id")
                if request.get("reply_to") != responses_topic:
                    response = {"correlation_id": cid, "success": False,
                                "error": "reply_to must be the configured response topic"}
                elif isinstance(cid, str) and cid in completed:
                    old_request, old_response = completed[cid]
                    response = old_response if old_request == request else {
                        "correlation_id": cid, "success": False,
                        "error": "correlation_id reused for a different request"}
                else:
                    log.info("WORKER PROCESS operation=%s correlation_id=%s",
                             request.get("operation"), cid)
                    response = process_request(request, notices)
                    if isinstance(cid, str) and cid.strip():
                        completed[cid] = (request, response)
                producer.send(responses_topic, response).get(timeout=15)
                log.info("WORKER REPLY topic=%s %s", responses_topic, response)
                consumer.commit()
    finally:
        consumer.close()
        producer.close()
```

A send_request function implemented in the hw7_authentication.py file. This acts as the Kafka client that creates the correlation ID, publishes the request and waits for the response.

```python
def send_request(operation, data, timeout=30):
    servers, requests_topic, responses_topic = kafka_settings()
    consumer = KafkaConsumer(bootstrap_servers=servers, group_id=None,
                             enable_auto_commit=False)
    producer = make_producer(servers)
    try:
        partitions = consumer.partitions_for_topic(responses_topic)
        if not partitions:
            raise RuntimeError("Response topic is unavailable; create both topics first")
        assigned = [TopicPartition(responses_topic, p) for p in sorted(partitions)]
        consumer.assign(assigned)
        consumer.seek_to_end(*assigned)
        for partition in assigned:
            consumer.position(partition)
        correlation_id = str(uuid4())
        request = {"correlation_id": correlation_id, "reply_to": responses_topic,
                   "operation": operation, "data": data}
        log.info("CLIENT SEND topic=%s %s", requests_topic, request)
        producer.send(requests_topic, request).get(timeout=15)
        deadline = time.monotonic() + timeout
        while time.monotonic() < deadline:
            for batch in consumer.poll(timeout_ms=500).values():
                for message in batch:
                    try:
                        response = json.loads(message.value)
                    except (ValueError, UnicodeDecodeError):
                        continue
                    if isinstance(response, dict) and response.get("correlation_id") == correlation_id:
                        log.info("CLIENT RECEIVE correlation preserved=True %s", response)
                        return response
        raise TimeoutError(f"No response for correlation_id={correlation_id}")
    finally:
        consumer.close()
        producer.close()
```

Implementation of the homework_demo.py file. Here CREATE is executed, the UUID is extracted, then GET is executed and finally the message is displayed.

```python
#Demonstration of CREATE then GET through the Kafka client.

import json
from uuid import uuid4

from hw7_authentication import send_request


def run_demo():
    #send_request creates one correlation ID, sends CREATE, and logs it.
    #worker logs processing and reply
    #send_request logs the matched response.
    created = send_request("CREATE_RECALL_NOTICE", {
        "name": "Spinach recall",
        "description": "Possible contamination in batch LOT-101",
        "unique_code": f"RECALL_{uuid4().hex[:12].upper()}",
        "quantity": 25,
        "related_grocery_recall_notice_id": 1,
    })
    if not created["success"]:
        raise RuntimeError(created["error"])
    # extract the generated UUID and send GET through the same client.
    retrieved = send_request("GET_RECALL_NOTICE", {
        "grocery_recall_notice_id": created["grocery_recall_notice_id"],
    })
    if not retrieved["success"]:
        raise RuntimeError(retrieved["error"])
    #send_request only returns a response matching its correlation ID.
    # Confirm GET returned the created entity and display its complete data.
    assert retrieved["grocery_recall_notice"]["recall_notice_id"] == created["recall_notice_id"]
    print(json.dumps(retrieved, indent=2))


if __name__ == "__main__":
    run_demo()
```

API wrap in the code. This is used to call the API request which uses the Kafka client (send_request).

```python
@router.post("/recall-notices", status_code=status.HTTP_201_CREATED)
def create_grocery_recall_notice(
    item: GroceryRecallNoticeCreate,
    user: User = Depends(require_roles(Role.writer, Role.moderator)),
):
    return api_request("CREATE_GROCERY_RECALL_NOTICE", item.model_dump())
```

Installing Kafka and setting up the PREFIX and the bootstrap server.

```
PS C:\Users\wei_1> cd 'C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code'
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python -m pip install
kafka-python
Collecting kafka-python
  Downloading kafka_python-3.0.12-py3-none-any.whl.metadata (11 kB)
Downloading kafka_python-3.0.12-py3-none-any.whl (614 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 614.3/614.3 kB 5.6 MB/s  0:00:00
Installing collected packages: kafka-python
Successfully installed kafka-python-3.0.12
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_PREFIX = "s
1339"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_BOOTSTRAP_S
ERVERS = "localhost:9092"
```

Starting up Docker services after building the containers. FastAPI service is seen successfully running at Port 127.0.0.1.9439

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose ps -a
NAME                COMMAND                  SERVICE             STATUS              PORTS
code-api-1          "uvicorn hw7_authent…"   api                 running             127.0.0.1:8439->8439/tcp
code-init-db-1      "python -c 'import h…"   init-db             exited (0)
code-init-topics-1  "/bin/bash -ec '/opt…"   init-topics         exited (0)
code-kafka-1        "/__cacert_entrypoin…"   kafka               running (healthy)   127.0.0.1:9092->9092/tcp
code-worker-1       "python hw7_authenti…"   worker              running
```

Services are shown to have started an connected successfully here and connections have been successfully established.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose logs -f worker
api
code-api-1     | INFO:     Started server process [1]
code-api-1     | INFO:     Waiting for application startup.
code-api-1     | INFO:     Application startup complete.
code-api-1     | INFO:     Uvicorn running on http://0.0.0.0:8439 (Press CTRL+C to quit)
code-worker-1  | 2026-11-02 07:30:48,960 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:
19092 <connecting> [IPv4 ('172.18.0.3', 19092)]>: connecting to kafka:19092 [('172.18.0.3', 19092) IPv4]
code-worker-1  | 2026-11-02 07:30:48,964 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:
19092 <checking_api_versions_recv> [IPv4 ('172.18.0.3', 19092)]>: Broker version identified as 2.6
code-worker-1  | 2026-11-02 07:30:48,965 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:
19092 <connected> [IPv4 ('172.18.0.3', 19092)]>: Connection complete.
```

The CREATE -> GET for the homework_demo.py file is first run in the terminal.

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose exec api python homework_demo.py
2026-11-02 08:10:02,910 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: con
necting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:02,914 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <checking_api_versions_recv> [IPv4 ('172.20.0.3
', 19092)]>: Broker version identified as 2.6
2026-11-02 08:10:02,914 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Conn
ection complete.
2026-11-02 08:10:02,917 <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>
: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:02,920 <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=kafka:19092 <checking_api_versions_recv> [IPv4 ('172.2
0.0.3', 19092)]>: Broker version identified as 2.6
2026-11-02 08:10:02,920 <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>:
 Connection complete.
2026-11-02 08:10:02,926 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: connecting to
 kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:02,927 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Connection com
plete.
2026-11-02 08:10:02,928 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Clos
ing connection.
2026-11-02 08:10:02,934 Resetting offset for partition TopicPartition(topic='s1339_entity_responses', partition=0) to offset 0.
2026-11-02 08:10:02,934 CLIENT SEND topic=s1339_entity_requests {'correlation_id': 'ad5c3379-bc45-4e21-8257-85e7cef855da', 'reply_to': 's1339_entity_respons
es', 'operation': 'CREATE_GROCERY_RECALL_NOTICE', 'data': {'name': 'Spinach recall', 'description': 'Possible contamination in batch LOT-101', 'unique_code'
: 'RECALL_B2C84B2D27A2', 'quantity': 25, 'related_entity_id': 1}}
2026-11-02 08:10:03,025 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: connecti
ng to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:03,025 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Connectio
n complete.
2026-11-02 08:10:03,025 <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>:
 Closing connection.
2026-11-02 08:10:03,057 CLIENT RECEIVE correlation preserved=True {'correlation_id': 'ad5c3379-bc45-4e21-8257-85e7cef855da', 'success': True, 'grocery_recal
l_notice_id': 'f8e05274-e938-404f-aedf-bd29f60b6869', 'message': 'Grocery recall notice created'}
2026-11-02 08:10:03,058 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Closing connec
tion.
2026-11-02 08:10:03,058 <KafkaProducer client_id=kafka-python-producer-1 transactional_id=None>: Closing the Kafka producer with 9223372036.0 secs timeout.
2026-11-02 08:10:03,059 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Closing c
onnection.
```

```
2026-11-02 08:10:03,061 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: con
necting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:03,065 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <checking_api_versions_recv> [IPv4 ('172.20.0.3
', 19092)]>: Broker version identified as 2.6
2026-11-02 08:10:03,065 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Conn
ection complete.
2026-11-02 08:10:03,067 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>
: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:03,071 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092 <checking_api_versions_recv> [IPv4 ('172.2
0.0.3', 19092)]>: Broker version identified as 2.6
2026-11-02 08:10:03,071 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>:
 Connection complete.
2026-11-02 08:10:03,077 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: connecting to
 kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:03,078 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Connection com
plete.
2026-11-02 08:10:03,078 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Clos
ing connection.
2026-11-02 08:10:03,081 Resetting offset for partition TopicPartition(topic='s1339_entity_responses', partition=0) to offset 1.
2026-11-02 08:10:03,081 CLIENT SEND topic=s1339_entity_requests {'correlation_id': '9a7097ed-1c2d-4b05-8447-f4b8f459d38c', 'reply_to': 's1339_entity_respons
es', 'operation': 'GET_GROCERY_RECALL_NOTICE', 'data': {'grocery_recall_notice_id': 'f8e05274-e938-404f-aedf-bd29f60b6869'}}
2026-11-02 08:10:03,176 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connecting> [IPv4 ('172.20.0.3', 19092)]>: connecti
ng to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:10:03,176 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Connectio
n complete.
2026-11-02 08:10:03,176 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>:
 Closing connection.
2026-11-02 08:10:03,188 CLIENT RECEIVE correlation preserved=True {'correlation_id': '9a7097ed-1c2d-4b05-8447-f4b8f459d38c', 'success': True, 'grocery_recal
l_notice': {'grocery_recall_notice_id': 'f8e05274-e938-404f-aedf-bd29f60b6869', 'name': 'Spinach recall', 'description': 'Possible contamination in batch LO
T-101', 'unique_code': 'RECALL_B2C84B2D27A2', 'quantity': 25, 'related_entity_id': 1}}
2026-11-02 08:10:03,188 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Closing connec
tion.
2026-11-02 08:10:03,189 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connected> [IPv4 ('172.20.0.3', 19092)]>: Closing c
onnection.
```

Output showing request with the Correlation ID


```
{
  "correlation_id": "9a7097ed-1c2d-4b05-8447-f4b8f459d38c",
  "success": true,
  "grocery_recall_notice": {
    "grocery_recall_notice_id": "f8e05274-e938-404f-aedf-bd29f60b6869",
    "name": "Spinach recall",
    "description": "Possible contamination in batch LOT-101",
    "unique_code": "RECALL_B2C84B2D27A2",
    "quantity": 25,
    "related_entity_id": 1
  }
}
```

Demonstration of CREATE -> GET with the homework_demo.py file. The output is logged to demo-evidence.txt

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose exec api python
 homework_demo.py 2>&1 | Tee-Object -FilePath demo-evidence.txt
docker : 2026-11-02 08:13:21,983 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092
<connecting> [IPv4 ('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
At line:1 char:1
+ docker compose exec api python homework_demo.py 2>&1 | Tee-Object -Fi ...
```

A CREATE_RECALL_NOTICE and GET_RECALL_NOTICE are generated from this command.

```
2026-11-02 08:13:22,003 CLIENT SEND topic=s1339_entity_requests {'correlation_id':
'f4a12f4d-493a-4b87-8a47-916f2c7258fb', 'reply_to': 's1339_entity_responses', 'operation':
'CREATE_GROCERY_RECALL_NOTICE', 'data': {'name': 'Spinach recall', 'description': 'Possible contamination in batch
LOT-101', 'unique_code': 'RECALL_6ECAC490D32C', 'quantity': 25, 'related_entity_id': 1}}
2026-11-02 08:13:22,098 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connecting>
[IPv4 ('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:13:22,099 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connected>
[IPv4 ('172.20.0.3', 19092)]>: Connection complete.
2026-11-02 08:13:22,099 <BrokerConnection client_id=kafka-python-producer-1, node_id=bootstrap-0 host=kafka:19092
<connected> [IPv4 ('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,112 CLIENT RECEIVE correlation preserved=True {'correlation_id':
'f4a12f4d-493a-4b87-8a47-916f2c7258fb', 'success': True, 'grocery_recall_notice_id':
'84c82fc8-7b98-45ea-a209-db0ca1b730ab', 'message': 'Grocery recall notice created'}
2026-11-02 08:13:22,112 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4
('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,113 <KafkaProducer client_id=kafka-python-producer-1 transactional_id=None>: Closing the Kafka
producer with 9223372036.0 secs timeout.
2026-11-02 08:13:22,113 <BrokerConnection client_id=kafka-python-producer-1, node_id=1 host=kafka:19092 <connected>
[IPv4 ('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,115 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092
<connecting> [IPv4 ('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:13:22,117 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092
<checking_api_versions_recv> [IPv4 ('172.20.0.3', 19092)]>: Broker version identified as 2.6
2026-11-02 08:13:22,118 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092
<connected> [IPv4 ('172.20.0.3', 19092)]>: Connection complete.
2026-11-02 08:13:22,119 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092
<connecting> [IPv4 ('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:13:22,122 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092
<checking_api_versions_recv> [IPv4 ('172.20.0.3', 19092)]>: Broker version identified as 2.6
2026-11-02 08:13:22,122 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092
<connected> [IPv4 ('172.20.0.3', 19092)]>: Connection complete.
2026-11-02 08:13:22,126 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connecting> [IPv4
('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:13:22,127 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4
('172.20.0.3', 19092)]>: Connection complete.
2026-11-02 08:13:22,127 <BrokerConnection client_id=kafka-python-2.3.3, node_id=bootstrap-0 host=kafka:19092
<connected> [IPv4 ('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,129 Resetting offset for partition TopicPartition(topic='s1339_entity_responses', partition=0) to
offset 3.
```

```
2026-11-02 08:13:22,129 CLIENT SEND topic=s1339_entity_requests {'correlation_id':
'4395c24a-a8fd-4101-8251-46988b19916b', 'reply_to': 's1339_entity_responses', 'operation':
'GET_GROCERY_RECALL_NOTICE', 'data': {'grocery_recall_notice_id': '84c82fc8-7b98-45ea-a209-db0ca1b730ab'}}
2026-11-02 08:13:22,226 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connecting>
[IPv4 ('172.20.0.3', 19092)]>: connecting to kafka:19092 [('172.20.0.3', 19092) IPv4]
2026-11-02 08:13:22,227 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connected>
[IPv4 ('172.20.0.3', 19092)]>: Connection complete.
2026-11-02 08:13:22,227 <BrokerConnection client_id=kafka-python-producer-2, node_id=bootstrap-0 host=kafka:19092
<connected> [IPv4 ('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,257 CLIENT RECEIVE correlation preserved=True {'correlation_id':
'4395c24a-a8fd-4101-8251-46988b19916b', 'success': True, 'grocery_recall_notice': {'grocery_recall_notice_id':
'84c82fc8-7b98-45ea-a209-db0ca1b730ab', 'name': 'Spinach recall', 'description': 'Possible contamination in batch
LOT-101', 'unique_code': 'RECALL_6ECAC490D32C', 'quantity': 25, 'related_entity_id': 1}}
2026-11-02 08:13:22,257 <BrokerConnection client_id=kafka-python-2.3.3, node_id=1 host=kafka:19092 <connected> [IPv4
('172.20.0.3', 19092)]>: Closing connection.
2026-11-02 08:13:22,259 <BrokerConnection client_id=kafka-python-producer-2, node_id=1 host=kafka:19092 <connected>
[IPv4 ('172.20.0.3', 19092)]>: Closing connection.
```

Request message shown at the end of the output.


```
{
  "correlation_id": "4395c24a-a8fd-4101-8251-46988b19916b",
  "success": true,
  "grocery_recall_notice": {
    "grocery_recall_notice_id": "84c82fc8-7b98-45ea-a209-db0ca1b730ab",
    "name": "Spinach recall",
    "description": "Possible contamination in batch LOT-101",
    "unique_code": "RECALL_6ECAC490D32C",
    "quantity": 25,
    "related_entity_id": 1
  }
}
```

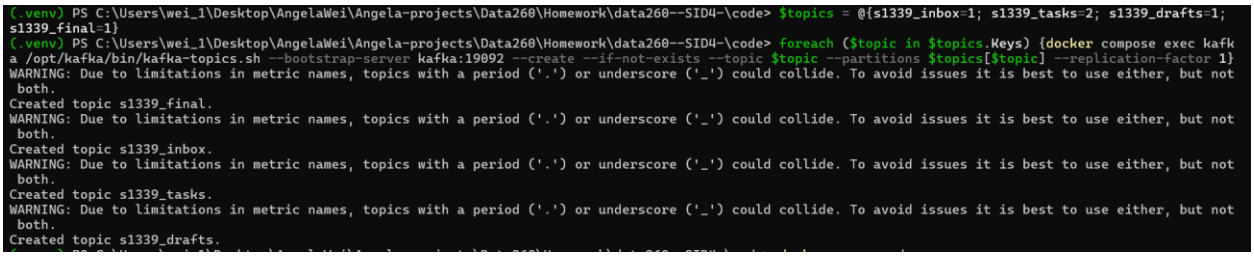

The output of the worker receiving, process and replying is logged to worker-evidence-txt .


```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose logs --timestamps worker | Select-String 'WORKER RE
CEIVE|WORKER PROCESS|WORKER REPLY' | Tee-Object -FilePath worker-evidence.txt

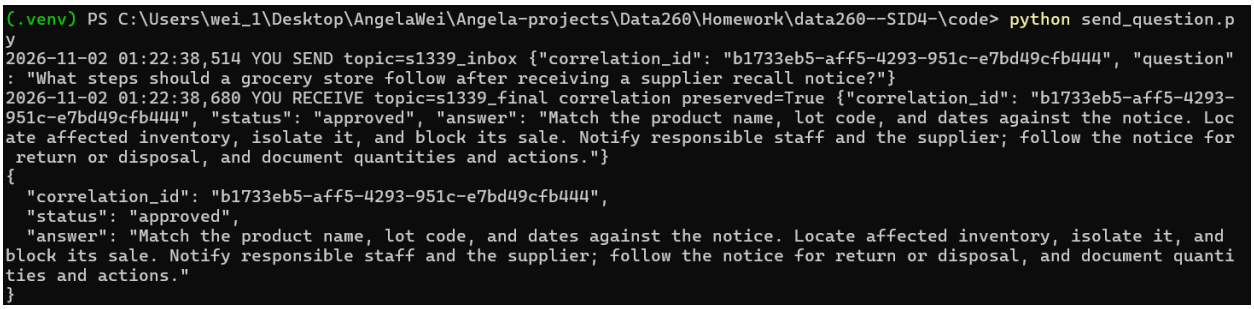
code-worker-1  | 2026-11-02T08:10:03.046726916Z 2026-11-02 08:10:03,046 WORKER RECEIVE {'correlation_id':
'ad5c3379-bc45-4e21-8257-85e7cef855da', 'reply_to': 's1339_entity_responses', 'operation':
'CREATE_GROCERY_RECALL_NOTICE', 'data': {'name': 'Spinach recall', 'description': 'Possible contamination in batch
LOT-101', 'unique_code': 'RECALL_B2C84B2D27A2', 'quantity': 25, 'related_entity_id': 1}}
code-worker-1  | 2026-11-02T08:10:03.046774737Z 2026-11-02 08:10:03,046 WORKER PROCESS
operation=CREATE_GROCERY_RECALL_NOTICE correlation_id=ad5c3379-bc45-4e21-8257-85e7cef855da
code-worker-1  | 2026-11-02T08:10:03.056443423Z 2026-11-02 08:10:03,056 WORKER REPLY topic=s1339_entity_responses
{'correlation_id': 'ad5c3379-bc45-4e21-8257-85e7cef855da', 'success': True, 'grocery_recall_notice_id':
'f8e05274-e938-404f-aedf-bd29f60b6869', 'message': 'Grocery recall notice created'}
code-worker-1  | 2026-11-02T08:10:03.183889741Z 2026-11-02 08:10:03,183 WORKER RECEIVE {'correlation_id':
'9a7097ed-1c2d-4b05-8447-f4b8f459d38c', 'reply_to': 's1339_entity_responses', 'operation':
'GET_GROCERY_RECALL_NOTICE', 'data': {'grocery_recall_notice_id': 'f8e05274-e938-404f-aedf-bd29f60b6869'}}
code-worker-1  | 2026-11-02T08:10:03.183924908Z 2026-11-02 08:10:03,183 WORKER PROCESS
operation=GET_GROCERY_RECALL_NOTICE correlation_id=9a7097ed-1c2d-4b05-8447-f4b8f459d38c
code-worker-1  | 2026-11-02T08:10:03.187523351Z 2026-11-02 08:10:03,186 WORKER REPLY topic=s1339_entity_responses
{'correlation_id': '9a7097ed-1c2d-4b05-8447-f4b8f459d38c', 'success': True, 'grocery_recall_notice':
{'grocery_recall_notice_id': 'f8e05274-e938-404f-aedf-bd29f60b6869', 'name': 'Spinach recall', 'description':
'Possible contamination in batch LOT-101', 'unique_code': 'RECALL_B2C84B2D27A2', 'quantity': 25, 'related_entity_id':
1}}
code-worker-1  | 2026-11-02T08:13:22.107818689Z 2026-11-02 08:13:22,107 WORKER RECEIVE {'correlation_id':
'f4a12f4d-493a-4b87-8a47-916f2c7258fb', 'reply_to': 's1339_entity_responses', 'operation':
'CREATE_GROCERY_RECALL_NOTICE', 'data': {'name': 'Spinach recall', 'description': 'Possible contamination in batch
LOT-101', 'unique_code': 'RECALL_6ECAC490D32C', 'quantity': 25, 'related_entity_id': 1}}
code-worker-1  | 2026-11-02T08:13:22.107908880Z 2026-11-02 08:13:22,107 WORKER PROCESS
operation=CREATE_GROCERY_RECALL_NOTICE correlation_id=f4a12f4d-493a-4b87-8a47-916f2c7258fb
code-worker-1  | 2026-11-02T08:13:22.111793495Z 2026-11-02 08:13:22,111 WORKER REPLY topic=s1339_entity_responses
{'correlation_id': 'f4a12f4d-493a-4b87-8a47-916f2c7258fb', 'success': True, 'grocery_recall_notice_id':
'84c82fc8-7b98-45ea-a209-db0ca1b730ab', 'message': 'Grocery recall notice created'}
code-worker-1  | 2026-11-02T08:13:22.243480151Z 2026-11-02 08:13:22,241 WORKER RECEIVE {'correlation_id':
'4395c24a-a8fd-4101-8251-46988b19916b', 'reply_to': 's1339_entity_responses', 'operation':
'GET_GROCERY_RECALL_NOTICE', 'data': {'grocery_recall_notice_id': '84c82fc8-7b98-45ea-a209-db0ca1b730ab'}}
code-worker-1  | 2026-11-02T08:13:22.243571667Z 2026-11-02 08:13:22,242 WORKER PROCESS
operation=GET_GROCERY_RECALL_NOTICE correlation_id=4395c24a-a8fd-4101-8251-46988b19916b
code-worker-1  | 2026-11-02T08:13:22.253357235Z 2026-11-02 08:13:22,251 WORKER REPLY topic=s1339_entity_responses
{'correlation_id': '4395c24a-a8fd-4101-8251-46988b19916b', 'success': True, 'grocery_recall_notice':
{'grocery_recall_notice_id': '84c82fc8-7b98-45ea-a209-db0ca1b730ab', 'name': 'Spinach recall', 'description':
'Possible contamination in batch LOT-101', 'unique_code': 'RECALL_6ECAC490D32C', 'quantity': 25, 'related_entity_id':
1}}
```

Part 2

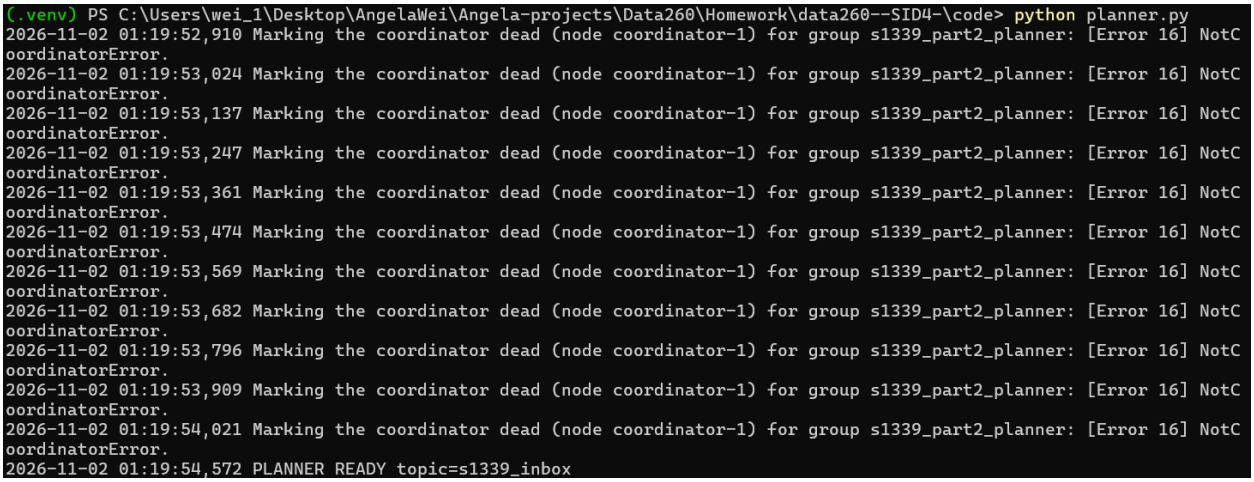
Running Kafka command to create topics.



Request is sent out from send_question.py



Planner.py successfully starting up in terminal and listening.



Planner.py successfully receiving request from send_question.py. A plan is then created to be sent to s1339 tasks.

```
2026-11-02 01:22:38,631 PLANNER RECEIVE topic=s1339_inbox {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "qu
estion": "What steps should a grocery store follow after receiving a supplier recall notice?"}
2026-11-02 01:22:38,631 PLANNER SEND topic=s1339_tasks {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."]}
```

Code in planner.py to complete this task.

```python
def make_plan(message):
    return {
        'correlation_id': message['correlation_id'],
        'question': message['question'],
        'plan': [
            'Identify the product and affected lot from the recall notice.',
            'Describe inventory isolation and sale blocking.',
            'Describe notification, disposition, and documentation.',
        ],
    }
```

Writer.py successfully starting up in terminal and listening. The request is then successfully received. The plan from s1339 task is made into a draft at s1339_drafts.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 01:20:52,164 WRITER READY topic=s1339_tasks
2026-11-02 01:22:38,647 WRITER RECEIVE topic=s1339_tasks {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "que
stion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the pro
duct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notificatio
n, disposition, and documentation."]}
2026-11-02 01:22:38,647 WRITER SEND topic=s1339_drafts {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locate aff
ected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return
 or disposal, and document quantities and actions."}
```

Code in writer.py to complete this task.

```python
def make_draft(message):
    #optional delay exposes the mid-processing crash window.
    delay = float(os.environ.get('WRITER_DELAY_SECONDS', '0'))
    if delay < 0 or delay > 120:
        raise ValueError('WRITER_DELAY_SECONDS must be between 0 and 120')
    log.info('WRITER PROCESS START pid=%s correlation_id=%s delay_seconds=%s',
             os.getpid(), message['correlation_id'], delay)
    time.sleep(delay)
    sections = {
        'Identify the product and affected lot from the recall notice.':
            'Match the product name, lot code, and dates against the notice.',
        'Describe inventory isolation and sale blocking.':
            'Locate affected inventory, isolate it, and block its sale.',
        'Describe notification, disposition, and documentation.':
            'Notify responsible staff and the supplier; follow the notice for '
            'return or disposal, and document quantities and actions.',
    }
    draft = ' '.join(sections[step] for step in message['plan'])
    return {
        'correlation_id': message['correlation_id'],
        'question': message['question'],
        'plan': message['plan'],
        'draft': draft,
    }
```

Reviewer.py successfully starting up in terminal and listening. The request is then successfully received. The draft from s1339_drafts is now used and the status and answer are now created and sent to s1339_final.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python reviewer.py
2026-11-02 01:21:33,475 REVIEWER READY topic=s1339_drafts
2026-11-02 01:22:38,664 REVIEWER RECEIVE topic=s1339_drafts {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "
question": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the
product and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notifica
tion, disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locat
e affected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for r
eturn or disposal, and document quantities and actions."}
2026-11-02 01:22:38,664 REVIEWER SEND topic=s1339_final {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "stat
us": "approved", "answer": "Match the product name, lot code, and dates against the notice. Locate affected inventory, i
solate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and
document quantities and actions."}
```

Code in reviewer.py to complete this task.

```python
def review(message):
    draft = message['draft']
    required = ('lot', 'isolate', 'block', 'notify', 'document')
    approved = all(word in draft.lower() for word in required)
    return {
        'correlation_id': message['correlation_id'],
        'status': 'approved' if approved else 'needs_revision',
        'answer': draft,
    }
```

The output seen after running send_question.py is the result of s1339_final.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python send_question.p
y
2026-11-02 01:22:38,514 YOU SEND topic=s1339_inbox {"correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444", "question"
: "What steps should a grocery store follow after receiving a supplier recall notice?"}
2026-11-02 01:22:38,680 YOU RECEIVE topic=s1339_final correlation preserved=True {"correlation_id": "b1733eb5-aff5-4293-
951c-e7bd49cfb444", "status": "approved", "answer": "Match the product name, lot code, and dates against the notice. Loc
ate affected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for
 return or disposal, and document quantities and actions."}
{
  "correlation_id": "b1733eb5-aff5-4293-951c-e7bd49cfb444",
  "status": "approved",
  "answer": "Match the product name, lot code, and dates against the notice. Locate affected inventory, isolate it, and
block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and document quanti
ties and actions."
}
```

Code in send_question.py to send out two questions and receive the final answers.

```python
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('question', nargs='?', default=
                        'What steps should a grocery store follow after '
                        'receiving a supplier recall notice?')
    args = parser.parse_args()
    final_topic = f'{PREFIX}_final'
    consumer = KafkaConsumer(bootstrap_servers=BROKER, group_id=None,
                             enable_auto_commit=False)
    client = producer()
    try:
        partitions = consumer.partitions_for_topic(final_topic)
        if not partitions:
            raise RuntimeError('Create the four Part 2 topics first')
        assigned = [TopicPartition(final_topic, p) for p in sorted(partitions)]
        consumer.assign(assigned)
        consumer.seek_to_end(*assigned)
        for partition in assigned:
            consumer.position(partition)
        cid = str(uuid4())
        publish(client, f'{PREFIX}_inbox',
                {'correlation_id': cid, 'question': args.question}, 'YOU')
        deadline = time.monotonic() + 60
        while time.monotonic() < deadline:
            for batch in consumer.poll(timeout_ms=500).values():
                for record in batch:
                    result = json.loads(record.value)
                    if result.get('correlation_id') == cid:
                        log.info('YOU RECEIVE topic=%s correlation preserved=True %s',
                                 final_topic, json.dumps(result))
                        print(json.dumps(result, indent=2))
                        return
        raise TimeoutError(f'No final response for correlation_id={cid}')
    finally:
        consumer.close()
        client.close()
```

Part 3

Duplicate delivery

The message sent out is a block of code that uses the same question twice with the same Correlation ID.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> @'
>> import json
>> from uuid import uuid4
>> from kafka import KafkaProducer
>>
>> producer = KafkaProducer(
>>     bootstrap_servers="127.0.0.1:9092",
>>     acks="all",
>>     value_serializer=lambda value: json.dumps(value).encode("utf-8"),
>> )
>>
>> message = {
>>     "correlation_id": str(uuid4()),
>>     "question": "What steps should a grocery store follow after receiving a supplier recall notice?"
>> }
>>
>> try:
>>     for delivery in (1, 2):
>>         producer.send(
>>             "s1339_inbox",
>>             key=message["correlation_id"].encode("utf-8"),
>>             value=message,
>>         ).get(timeout=20)
>>         print(f"Delivery {delivery}: {json.dumps(message)}")
>> finally:
>>     producer.close()
>> '@ | python -
Delivery 1: {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "question": "What steps should a grocery store follow after receiving a supplier reca
ll notice?"}
Delivery 2: {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "question": "What steps should a grocery store follow after receiving a supplier reca
ll notice?"}
```

In planner:

```
2026-11-02 01:34:19,564 PLANNER RECEIVE topic=s1339_inbox {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "qu
estion": "What steps should a grocery store follow after receiving a supplier recall notice?"}
2026-11-02 01:34:19,564 PLANNER SEND topic=s1339_tasks {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."]}
2026-11-02 01:34:19,591 PLANNER RECEIVE topic=s1339_inbox {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "qu
estion": "What steps should a grocery store follow after receiving a supplier recall notice?"}
2026-11-02 01:34:19,591 PLANNER SEND topic=s1339_tasks {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."]}
```

Output shows the two questions being processed at once.

In writer:

```
2026-11-02 01:34:19,572 WRITER RECEIVE topic=s1339_tasks {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "que
stion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the pro
duct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notificatio
n, disposition, and documentation."]}
2026-11-02 01:34:19,573 WRITER SEND topic=s1339_drafts {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locate aff
ected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return
 or disposal, and document quantities and actions."}
2026-11-02 01:34:19,596 WRITER RECEIVE topic=s1339_tasks {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "que
stion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the pro
duct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notificatio
n, disposition, and documentation."]}
2026-11-02 01:34:19,596 WRITER SEND topic=s1339_drafts {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "quest
ion": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the produ
ct and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notification,
 disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locate aff
ected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return
 or disposal, and document quantities and actions."}
```

Output again is being shown as the two questions being processed at once.

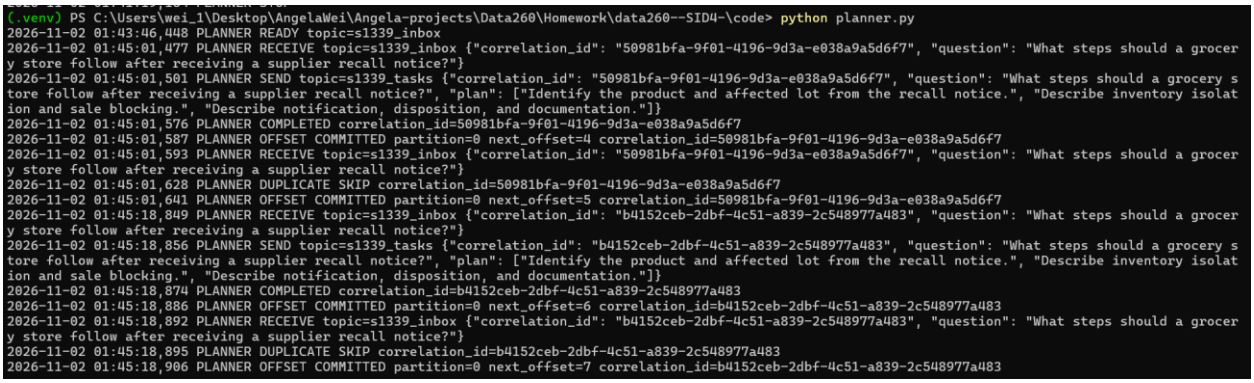
In reviewer:

```
2026-11-02 01:34:19,580 REVIEWER RECEIVE topic=s1339_drafts {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "
question": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the
product and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notifica
tion, disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locat
e affected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for r
eturn or disposal, and document quantities and actions."}
2026-11-02 01:34:19,580 REVIEWER SEND topic=s1339_final {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "stat
us": "approved", "answer": "Match the product name, lot code, and dates against the notice. Locate affected inventory, i
solate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and
document quantities and actions."}
2026-11-02 01:34:19,603 REVIEWER RECEIVE topic=s1339_drafts {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "
question": "What steps should a grocery store follow after receiving a supplier recall notice?", "plan": ["Identify the
product and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe notifica
tion, disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locat
e affected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for r
eturn or disposal, and document quantities and actions."}
2026-11-02 01:34:19,603 REVIEWER SEND topic=s1339_final {"correlation_id": "992f4f2c-d25b-4ec8-a8af-bfc27a5f3f1e", "stat
us": "approved", "answer": "Match the product name, lot code, and dates against the notice. Locate affected inventory, i
solate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and
document quantities and actions."}
```

Output yet again, is still being shown as the two questions being processed at once.

Duplicate delivery with Idempotency

In planner:

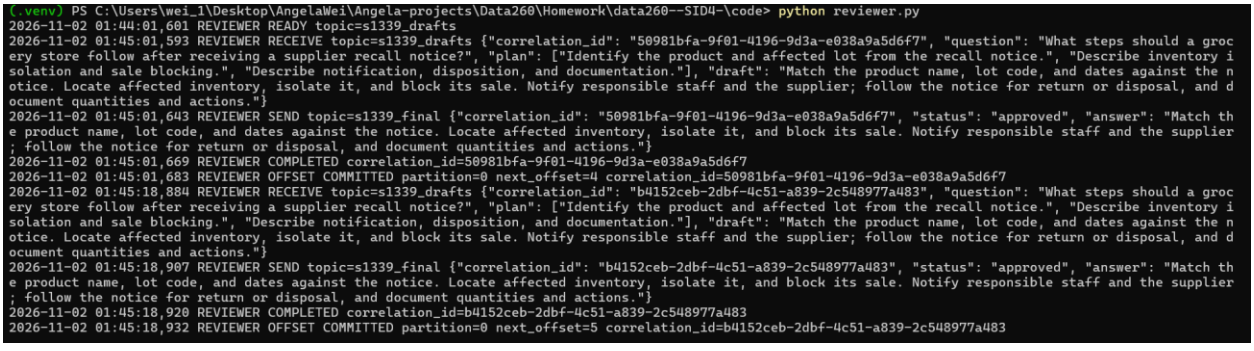


Duplicate is skipped upon running with idempotency.

In writer:



In reviewer:



Idempotency code implemented in planner, writer and reviewer for this functionality.

```
# persist completion separately for each prefix, stage and correlation id
IDEMPOTENCY_DB = os.environ.get(
    'IDEMPOTENCY_DB', str(Path(__file__).with_name('part3-idempotency.sqlite3'))
)
```

Consumer Death

A 45 second delay is set up before writer.py is run. This allows time for the process to be stopped before it is finished with processing.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:WRITER_DELAY_SECONDS = "45"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 01:56:27,451 WRITER READY topic=s1339_tasks
2026-11-02 01:57:20,546 WRITER RECEIVE topic=s1339_tasks {"correlation_id": "aab0493f-d958-470c-80c5-560a0ac4f073", "question": "What steps should a grocery
 store follow after receiving a supplier recall notice?", "plan": ["Identify the product and affected lot from the recall notice.", "Describe inventory isol
ation and sale blocking.", "Describe notification, disposition, and documentation."]}
2026-11-02 01:57:20,557 WRITER PROCESS START pid=29388 correlation_id=aab0493f-d958-470c-80c5-560a0ac4f073 delay_seconds=45.0
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code>
```

The PID is entered in the Stop-Process command after it is generated when writer.py starts running. Entering this command in a separate terminal will stop writer.py mid-process.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> Stop-Process -Id 29388
 -Force
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code>
```

A timeout error is now seen in the writer.py terminal.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python send_question.py
2026-11-02 01:57:20,431 YOU SEND topic=s1339_inbox {"correlation_id": "aab0493f-d958-470c-80c5-560a0ac4f073", "question": "What steps should a grocery store
 follow after receiving a supplier recall notice?"}
Traceback (most recent call last):
  File "C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\send_question.py", line 77, in <module>
    main()
  File "C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code\send_question.py", line 70, in main
    raise TimeoutError(f'No final response for correlation_id={cid}')
TimeoutError: No final response for correlation_id=aab0493f-d958-470c-80c5-560a0ac4f073
```

After starting writer.py back up without any delays in place. Process is resumed and completes itself successfully.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 02:00:11,428 WRITER READY topic=s1339_tasks
2026-11-02 02:00:11,428 WRITER RECEIVE topic=s1339_tasks {"correlation_id": "aab0493f-d958-470c-80c5-560a0ac4f073", "question": "What steps should a grocery
 store follow after receiving a supplier recall notice?", "plan": ["Identify the product and affected lot from the recall notice.", "Describe inventory isol
ation and sale blocking.", "Describe notification, disposition, and documentation."]}
2026-11-02 02:00:11,432 WRITER PROCESS START pid=24832 correlation_id=aab0493f-d958-470c-80c5-560a0ac4f073 delay_seconds=0.0
2026-11-02 02:00:11,432 WRITER SEND topic=s1339_drafts {"correlation_id": "aab0493f-d958-470c-80c5-560a0ac4f073", "question": "What steps should a grocery s
tore follow after receiving a supplier recall notice?", "plan": ["Identify the product and affected lot from the recall notice.", "Describe inventory isolat
ion and sale blocking.", "Describe notification, disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice
. Locate affected inventory, isolate it, and block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and docume
nt quantities and actions."}
2026-11-02 02:00:11,448 WRITER COMPLETED correlation_id=aab0493f-d958-470c-80c5-560a0ac4f073
2026-11-02 02:00:11,461 WRITER OFFSET COMMITTED partition=0 next_offset=6 correlation_id=aab0493f-d958-470c-80c5-560a0ac4f073
```

Line of code in writer.py that allows for the delay

```
#optional delay exposes the mid-processing crash window.
delay = float(os.environ.get('WRITER_DELAY_SECONDS', '0'))
if delay < 0 or delay > 120:
    raise ValueError('WRITER_DELAY_SECONDS must be between 0 and 120')
log.info('WRITER PROCESS START pid=%s correlation_id=%s delay_seconds=%s',
         os.getpid(), message['correlation_id'], delay)
time.sleep(delay)
```

Poison Message

Command is run to create a dead letter topic.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose exec ka
fka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:19092 --create --if-not-exists --topic s1339_dead_letter --p
artitions 1 --replication-factor 1
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issu
es it is best to use either, but not both.
Created topic s1339_dead_letter.
```

The block of code is run in the terminal using a “bad” message.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> @'
>> import json
>> from uuid import uuid4
>> from kafka import KafkaProducer
>>
>> producer = KafkaProducer(
>>     bootstrap_servers="127.0.0.1:9092",
>>     acks="all",
>>     value_serializer=lambda value: json.dumps(value).encode("utf-8"),
>> )
>>
>> # Invalid: Writer requires a plan, which this message omits.
>> bad = {
>>     "correlation_id": str(uuid4()),
>>     "question": "Poison-message test"
>> }
>>
>> good = {
>>     "correlation_id": str(uuid4()),
>>     "question": "What should a grocery store do after a recall?",
>>     "plan": [
>>         "Identify the product and affected lot from the recall notice.",
>>         "Describe inventory isolation and sale blocking.",
>>         "Describe notification, disposition, and documentation."
>>     ]
>> }
>>
>> try:
>>     for label, message in (("BAD", bad), ("GOOD", good)):
>>         producer.send(
>>             "s1339_tasks",
>>             partition=0,
>>             value=message,
>>             key=message["correlation_id"].encode("utf-8"),
>>         ).get(timeout=20)
>>         print(label, json.dumps(message))
>> finally:
>>     producer.close()
>> '@ | python -
BAD {"correlation_id": "804edef2-6721-4ab7-8a73-a2be38d56112", "question": "Poison-message test"}
GOOD {"correlation_id": "1671d1e5-c002-4c79-8c91-1bff56edf05d", "question": "What should a grocery store do after a recall?", "plan": ["Identify the product
```

Writer.py will commit this “bad” message to offset. It is then moved to dead-letter topic.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 02:07:12,556 WRITER READY topic=s1339_tasks
2026-11-02 02:08:11,531 WRITER RECEIVE topic=s1339_tasks partition=0 offset=6 raw={"correlation_id": "804edef2-6721-4ab7-8a73-a2be38d56112", "question": "Po
ison-message test"}
2026-11-02 02:08:11,531 WRITER ATTEMPT 1/3 topic=s1339_tasks partition=0 offset=6 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112
2026-11-02 02:08:11,561 WRITER PROCESS START pid=16356 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 delay_seconds=0.0
2026-11-02 02:08:11,562 WRITER ATTEMPT FAILED 1/3 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 error='plan'
2026-11-02 02:08:12,564 WRITER ATTEMPT 2/3 topic=s1339_tasks partition=0 offset=6 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112
2026-11-02 02:08:12,565 WRITER PROCESS START pid=16356 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 delay_seconds=0.0
2026-11-02 02:08:12,566 WRITER ATTEMPT FAILED 2/3 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 error='plan'
2026-11-02 02:08:13,567 WRITER ATTEMPT 3/3 topic=s1339_tasks partition=0 offset=6 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112
2026-11-02 02:08:13,568 WRITER PROCESS START pid=16356 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 delay_seconds=0.0
2026-11-02 02:08:13,568 WRITER ATTEMPT FAILED 3/3 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112 error='plan'
2026-11-02 02:08:13,583 WRITER DEAD LETTER ACK topic=s1339_dead_letter {"correlation_id": "804edef2-6721-4ab7-8a73-a2be38d56112", "stage": "WRITER", "source
_topic": "s1339_tasks", "source_partition": 0, "source_offset": 6, "attempts": 3, "error": "KeyError: 'plan'", "original_message_base64": "eyJjb3JyZWxhdGlvb
l9pZCI6ICI4MDRlZGVmMi02NzIxLTRhYjctOGE3My1hMmJlMzhkNTYxMTIiLCAicXVlc3Rpb24iOiAiUG9pc29uLW1lc3NhZ2UgdGVzdCJ9", "original_message_text": "{\"correlation_id\":
 \"804edef2-6721-4ab7-8a73-a2be38d56112\", \"question\": \"Poison-message test\"}"}
2026-11-02 02:08:13,593 WRITER OFFSET COMMITTED partition=0 next_offset=7 correlation_id=804edef2-6721-4ab7-8a73-a2be38d56112
2026-11-02 02:08:13,599 WRITER RECEIVE topic=s1339_tasks partition=0 offset=7 raw={"correlation_id": "1671d1e5-c002-4c79-8c91-1bff56edf05d", "question": "Wh
at should a grocery store do after a recall?", "plan": ["Identify the product and affected lot from the recall notice.", "Describe inventory isolation and s
ale blocking.", "Describe notification, disposition, and documentation."]}
2026-11-02 02:08:13,599 WRITER ATTEMPT 1/3 topic=s1339_tasks partition=0 offset=7 correlation_id=1671d1e5-c002-4c79-8c91-1bff56edf05d
2026-11-02 02:08:13,601 WRITER PROCESS START pid=16356 correlation_id=1671d1e5-c002-4c79-8c91-1bff56edf05d delay_seconds=0.0
2026-11-02 02:08:13,601 WRITER SEND topic=s1339_drafts {"correlation_id": "1671d1e5-c002-4c79-8c91-1bff56edf05d", "question": "What should a grocery store d
o after a recall?", "plan": ["Identify the product and affected lot from the recall notice.", "Describe inventory isolation and sale blocking.", "Describe n
otification, disposition, and documentation."], "draft": "Match the product name, lot code, and dates against the notice. Locate affected inventory, isolate
 it, and block its sale. Notify responsible staff and the supplier; follow the notice for return or disposal, and document quantities and actions."}
2026-11-02 02:08:13,702 WRITER COMPLETED correlation_id=1671d1e5-c002-4c79-8c91-1bff56edf05d
2026-11-02 02:08:13,714 WRITER OFFSET COMMITTED partition=0 next_offset=8 correlation_id=1671d1e5-c002-4c79-8c91-1bff56edf05d
```

Final output of the dead-letter topic with the “bad” message.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose exec ka
fka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka:19092 --topic s1339_dead_letter --from-beginning -
--max-messages 1
{"correlation_id": "804edef2-6721-4ab7-8a73-a2be38d56112", "stage": "WRITER", "source_topic": "s1339_tasks", "source_par
tition": 0, "source_offset": 6, "attempts": 3, "error": "KeyError: 'plan'", "original_message_base64": "eyJjb3JyZWxhdGlv
bl9pZCI6ICI4MDRlZGVmMi02NzIxLTRhYjctOGE3My1hMmJlMzhkNTYxMTIiLCAicXVlc3Rpb24iOiAiUG9pc29uLW1lc3NhZ2UgdGVzdCJ9", "original
_message_text": "{\"correlation_id\": \"804edef2-6721-4ab7-8a73-a2be38d56112\", \"question\": \"Poison-message test\"}"}
Processed a total of 1 messages
```

Code for dead letter topic in planner, writer and reviewer.

```python
    #preserve the raw input
    dead_letter = {
        'correlation_id': cid,
        'stage': stage,
        'source_topic': record.topic,
        'source_partition': record.partition,
        'source_offset': record.offset,
        'attempts': attempts,
        'error': last_error,
        'original_message_base64': base64.b64encode(record.value).decode('ascii'),
        'original_message_text': record.value.decode('utf-8', errors='replace'),
    }
    topic = f'{PREFIX}_dead_letter'
    key = cid.encode('utf-8') if isinstance(cid, str) else None
    #do not return to the input-offset commit until the dead letter send succeeds
    client.send(topic, value=dead_letter, key=key).get(timeout=20)
    log.info('%s DEAD LETTER ACK topic=%s %s', stage, topic,
             json.dumps(dead_letter))
    return cid
```

Ordering

Generating 20 messages using VERIFY_SEED

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> docker compose exec ka
fka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:19092 --describe --topic s1339_tasks
Topic: s1339_tasks      TopicId: RKOjiSp9RJmkbe5w3jD-rQ PartitionCount: 2       ReplicationFactor: 1    Configs: min.ins
ync.replicas=1
        Topic: s1339_tasks      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: s1339_tasks      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
```

First terminal running writer.py

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_PREFIX = "s1339"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_BOOTSTRAP_SERVERS = "127.0.0.1:9092"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:WRITER_DELAY_SECONDS = "0"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:ORDERING_PARTITION0_DELAY = "1"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 02:17:43,029 WRITER ASSIGNMENT pid=17912 group=s1339_part2_writer partitions=(0, 1)
2026-11-02 02:17:43,029 WRITER READY topic=s1339_tasks
2026-11-02 02:18:42,466 Heartbeat failed for group s1339_part2_writer because it is rebalancing
2026-11-02 02:18:43,409 WRITER ASSIGNMENT pid=17912 group=s1339_part2_writer partitions=(0,)
```

Second terminal running writer.py

```
PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_PREFIX = "s1339"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:KAFKA_BOOTSTRAP_SERVERS = "127.0.0.1:9092"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:WRITER_DELAY_SECONDS = "0"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> $env:ORDERING_PARTITION0_DELAY = "1"
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python writer.py
2026-11-02 02:18:41,217 WRITER ASSIGNMENT pid=12140 group=s1339_part2_writer partitions=()
2026-11-02 02:18:43,221 WRITER ASSIGNMENT pid=12140 group=s1339_part2_writer partitions=(1,)
2026-11-02 02:18:43,221 WRITER READY topic=s1339_tasks
```

The messages are sent to these terminals.

Successful unkeyed test run in third terminal for both terminals running writer.py

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python ordering_test.p
y unkeyed 2>&1 | Tee-Object -FilePath ordering-unkeyed-log.txt
```

```
Received 20/20; Writer PIDs=[12140, 17912]; saved C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data
260--SID4-\code\ordering-unkeyed-results.json
Unkeyed run observed per-notice reordering: True
```

Successful unkeyed test run in third terminal for both terminals running writer.py. Here, grocery_recall_notice_id is used as the key.

```
(.venv) PS C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data260--SID4-\code> python ordering_test.p
y keyed 2>&1 | Tee-Object -FilePath ordering-keyed-log.txt
```

```
Received 20/20; Writer PIDs=[12140, 17912]; saved C:\Users\wei_1\Desktop\AngelaWei\Angela-projects\Data260\Homework\data
260--SID4-\code\ordering-keyed-results.json
PASS: every notice stayed in one partition and preserved its order.
```

Comparison

In Homework 2, all 3 agents were coordinated together using a LangGraph graph, and everything was completed through a single process. The planner and the reviewer would return updates to share the graph state and the graph edges were responsible for the next step in execution.

The Kafka implementation of having all three agents in separate files allowed them to run independent of one another and communicate by exchange JSON messages as outputs.

Two benefits that were gained by moving to the Kafka format include independent scaling as shown by the two writer processes in Ordering sharing the partitions and failure isolation, as demonstrated by writer in Consumer Death when the process restarted after it was manually terminated.

Two new responsibilities gained in this new Kafka implementation include preventing side effects and preserving the message orders.

Completed Failure Table:

Failure scenario	Before fix	After fix
Duplicate delivery	Two questions with the same correlation_id were processed at once.	Adding idempotency helped the duplicate message be skipped, allowing only the first question to be successfully processed.
Consumer killed mid-work	Command entered to second terminal containing PID generate by writer while it is running is entered to stop writer mid process	Restarting writer will allow the terminated process to successfully resume and complete run.
Poison message	Message is marked as a bad message and retried to a limit	The "bad" message is sent to dead letter topic
Two partitions / two consumers	Unkeyed test is successfully run and per-notice reordering is true	Keyed test is successfully run and passed: key is grocery_recall_notice_id

Github:

https://github.com/Aywei2001/data260--SID4-